

Lab Assignment No.	09
Title	Implement Ant colony optimization by solving the Traveling salesman problem using python Problem statement- A salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 09

Title: Implementation of Ant Colony Optimization for Solving the Traveling Salesman Problem Using Python

Problem Statement: A salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city, using Ant Colony Optimization (ACO) techniques.

Prerequisites:

- Basic knowledge of Python programming
- Understanding of optimization techniques and combinatorial problems
- Familiarity with metaheuristic algorithms, specifically Ant Colony Optimization
- Basic knowledge of Jupyter Notebook environment

Software Requirements:

- Jupyter Notebook
- Python 3.x
- Required Python libraries: NumPy, Matplotlib, NetworkX (for visualization of paths)

Hardware Requirements:

- A computer with minimum 4 GB RAM
- Dual-core processor or higher
- Stable internet connection

Theory: Ant Colony Optimization (ACO) is a probabilistic technique inspired by the foraging behavior of ants in nature. Ants find the shortest path between their colony and food sources by depositing pheromones along their trails. Over time, shorter paths accumulate more pheromones, attracting more ants and reinforcing the optimal route.

In the context of the Traveling Salesman Problem (TSP), ACO is used to simulate the behavior of artificial ants that explore different routes between cities. These ants communicate indirectly through pheromone trails, which guide the search process towards optimal or near-optimal solutions.

Key Concepts:

1. **Pheromone Trail (τ):** A numerical value representing the desirability of a path between two cities. Higher pheromone levels indicate preferred routes.
2. **Heuristic Information (η):** Represents the visibility or attractiveness of a path, often inversely proportional to the distance between cities.
3. **Probability Transition Rule:** Determines the likelihood of an ant choosing a specific path based on pheromone concentration and heuristic information.
4. **Pheromone Update Rule:** Involves evaporation (reducing pheromone intensity over time) and deposition (adding pheromones based on the quality of the solution).
5. **Exploration vs. Exploitation:** Balances between exploring new paths and exploiting known good routes to optimize the solution.

Applications:

- Routing and logistics optimization
- Network design and resource allocation
- Scheduling problems
- Vehicle routing in transportation systems

Advantages of ACO:

- Efficient for large-scale problems
- Adaptive to dynamic environments
- Balances exploration and exploitation

Challenges:

- Sensitive to parameter tuning
- Risk of premature convergence
- Pathfinding in robotics and AI

Source Code –

```
# Import Required Libraries
```

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# Define coordinates for each city (for visualization purposes)
```

```
city_coordinates = np.array([  
    [0, 0],      # City 0  
    [10, 0],     # City 1  
    [10, 10],    # City 2  
    [0, 10]      # City 3  
])
```

```
# Define the distance matrix (distances between cities)
```

```
distance_matrix = np.array([  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
])
```

```
# Parameters for Ant Colony Optimization
```

```
num_ants = 10
```

```
num_iterations = 50
```

```
evaporation_rate = 0.5
```

```
pheromone_constant = 1.0
```

```
heuristic_constant = 1.0
```

```
# Initialize pheromone matrix and visibility matrix

num_cities = len(distance_matrix)

pheromone = np.ones((num_cities, num_cities)) # Pheromone matrix

# Handle division by zero in visibility matrix (replace 0s with infinity)

Visibility = np.where(distance_matrix == 0, np.inf, 1 / distance_matrix)

# ACO algorithm

For iteration in range(num_iterations):

    ant_routes = []

    for ant in range(num_ants):

        current_city = random.randint(0, num_cities - 1)

        visited_cities = [current_city]

        route = [current_city]

        while len(visited_cities) < num_cities:

            probabilities = []

            for city in range(num_cities):

                if city not in visited_cities:

                    pheromone_value = pheromone[current_city][city]

                    visibility_value = visibility[current_city][city]

                    probability = (pheromone_value ** pheromone_constant) * (visibility_value **
                        heuristic_constant)

                    probabilities.append((city, probability))

            probabilities = sorted(probabilities, key=lambda x: x[1], reverse=True)

            selected_city = probabilities[0][0]

            route.append(selected_city)

            visited_cities.append(selected_city)

            current_city = selected_city
```

```
    ant_routes.append(route)

# Update pheromone levels

delta_pheromone = np.zeros((num_cities, num_cities))

    for ant, route in enumerate(ant_routes):

        for I in range(len(route) - 1):
            city_a = route[i]

            city_b = route[I + 1]

delta_pheromone[city_a][city_b] += 1 / distance_matrix[city_a][city_b]

delta_pheromone[city_b][city_a] += 1/distance_matrix[city_a][city_b]

pheromone = (1 -evaporation_rate) * pheromone + delta_pheromone

# Find the best route

best_route_index = np.argmax([sum(distance_matrix[cities[i]][cities[(I + 1)% num_cities]]
    for I in range(num_cities

best_route = ant_routes[best_route_index]

shortest_distance = sum(distance_matrix[best_route[i]][best_route[(I + 1) % num_cities]]
    for I in range(num_cities

print("Best route:", best_route)

print("Shortest distance:", shortest_distance)

# Visualize the best route using city coordinates

best_route_coords = city_coordinates[best_route] # Get the coordinates of the best route

best_route_coords = np.vstack([best_route_coords, best_route_coords[0]]) # Add the starting
city at the end to close

x, y = zip(*best_route_coords) # Unpack coordinates

plt.plot(x, y, 'o-', color='blue')

plt.title(f"Best Route with Length: {shortest_distance}")

plt.xlabel('X')

plt.ylabel('Y')
```

plt.show()

Conclusion: Ant Colony Optimization provides an effective, nature-inspired approach to solving the Traveling Salesman Problem. By simulating the behavior of ants and leveraging pheromone-based communication, ACO efficiently explores possible routes, balancing exploration of new paths and exploitation of known good paths. The algorithm's flexibility and robustness make it suitable for complex optimization problems beyond TSP.